



Bubble & Selection Sort

Dr. Obed Noé Sámano Abonce



Sorting Algorithms

Introduction

What are they?

- They are sequences of instructions whose function is to accommodate the values of an array from lowest to highest.

What are they used for?

- To be able to search for a value more quickly.

Algorithm efficiency

Comparisons and movements

Execution-time efficiency can be measured by:

- Counting the number of **comparisons** (C)
- Counting the number of element **movements** (M)

Both as a function of
the input size



An algorithm is said to perform $O(n^2)$ comparisons when you compare n times the n elements, $n \times n = n^2$

Sorting Algorithms

Examples

- **Bubble Sort**
- **Selection Sort**
- Insertion sort
- Merge Sort
- Quicksort
- Heap Sort

Bubble Sort

Bubble Sort

The idea

- Compare **neighbours**.
 - If the left one is bigger, swap them.
 - Walk to the end.
 - The process must be repeated until there is no exchange.
-
- On every pass, the largest value still out of place **floats to the end**.



Bubble Sort

After each pass, what do we know?

After each pass, what do we know?

Example: after the first pass ($k=1$):

?	?	?	?	?	42
---	---	---	---	---	----

After pass **k**, the **last k** elements are in their final position.
This lets us reduce the number of comparisons in each pass.

Bubble Sort

Dance



<https://youtu.be/LriMvv9qDrk?si=xkT0JUYGV2ZmG18w>

Bubble Sort

Activity

Activity:

- 8 volunteers in front of the class
- The rest of the class count the number of comparisons and swaps

Bubble Sort

Trace example

 being compared
 just moved / swapped
 compared, left alone
 smallest so far (selection)
 final position



Tecnológico
de Monterrey

1 Pass 1 7 comparisons • 7 swaps

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
17	42	5	30	8	29	3	15
17	5	42	30	8	29	3	15
17	5	30	42	8	29	3	15
17	5	30	8	42	29	3	15
17	5	30	8	29	42	3	15
17	5	30	8	29	3	42	15
17	5	30	8	29	3	15	42

2 Pass 2 6 comparisons • 5 swaps

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
5	17	30	8	29	3	15	42
5	17	30	8	29	3	15	42
5	17	8	30	29	3	15	42
5	17	8	29	30	3	15	42
5	17	8	29	3	30	15	42
5	17	8	29	3	15	30	42
5	17	8	29	3	15	30	42

3 Pass 3 5 comparisons • 3 swaps

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
5	17	8	29	3	15	30	42
5	8	17	29	3	15	30	42
5	8	17	29	3	15	30	42
5	8	17	3	29	15	30	42
5	8	17	3	15	29	30	42
5	8	17	3	15	29	30	42
5	8	17	3	15	29	30	42

4 Pass 4 4 comparisons • 2 swaps

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
5	8	17	3	15	29	30	42
5	8	17	3	15	29	30	42
5	8	3	17	15	29	30	42
5	8	3	15	17	29	30	42

5 Pass 5 3 comparisons • 1 swap

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
5	8	3	15	17	29	30	42
5	3	8	15	17	29	30	42
5	3	8	15	17	29	30	42

6 Pass 6 2 comparisons • 1 swap

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
3	5	8	15	17	29	30	42
3	5	8	15	17	29	30	42

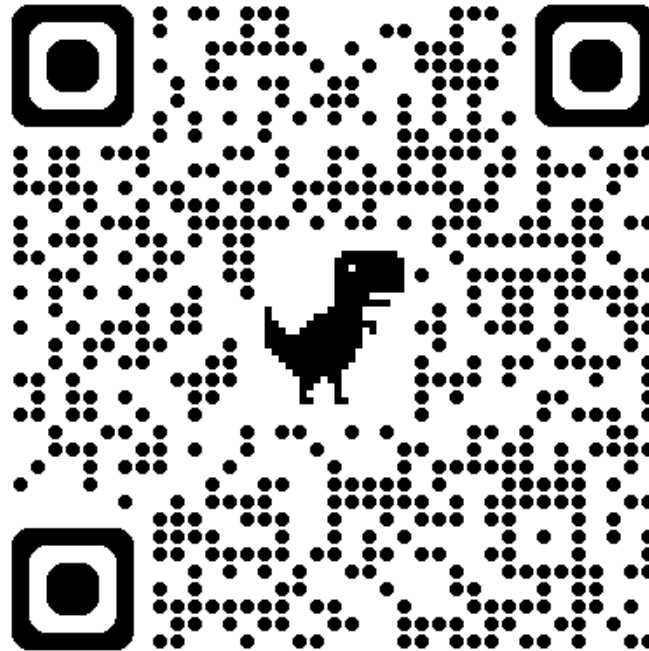
7 Pass 7 1 comparison • 0 swaps

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
3	5	8	15	17	29	30	42

Total: 28 comparisons • 19 swaps

Bubble Sort

Visualization 1



<https://noe-tec.github.io/data-structures/01-fundamentals/pages/sorting-walkthrough.html>

Bubble Sort

The code

```
void bubbleSort(int a[], int n) {
    bool swapped = true;
    for (int pass = 0; pass < n - 1 && swapped; pass++) {
        swapped = false;
        for (int j = 0; j < n - 1 - pass; j++) {
            if (a[j + 1] < a[j]) {
                int tmp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = tmp;
                swapped = true;
            }
        }
    }
}
```

- **Best case:**
 - A list already in order
 - A single pass
 - $O(n)$
- **Worst case:**
 - $O(n^2)$

Selection Sort

Selection Sort

The idea

- Look at everything that is left.
- Find the **smallest**.
- Put it where it belongs. **One swap**.
- Repeat with what remains.

Selection Sort

After each pass, what do we know?

After each pass, what do we know?

Example: after the first pass ($k=2$):

3	5	?	?	?	?
---	---	---	---	---	---

After pass k , the **first k** elements are fixed and already in order.
This lets us reduce the number of comparisons in each pass.

Selection Sort

Selection Dance



<https://youtu.be/LriMvv9qDrk?si=xkT0JUYGV2ZmG18w>

Selection Sort

Activity

Activity:

- 8 volunteers in front of the class
- The rest of the class count the number of comparisons and swaps

Selection Sort

Trace example

■ being compared
 ■ just moved / swapped
 ■ compared, left alone
 ■ smallest so far
 ■ final position

1 Pass 1 7 comparisons · swap $a[0] \leftrightarrow a[6]$

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
42	17	5	30	8	29	3	15
42	17	5	30	8	29	3	15
42	17	5	30	8	29	3	15
42	17	5	30	8	29	3	15
42	17	5	30	8	29	3	15
42	17	5	30	8	29	3	15
42	17	5	30	8	29	3	15
3	17	5	30	8	29	42	15

2 Pass 2 6 comparisons · swap $a[1] \leftrightarrow a[2]$

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
3	17	5	30	8	29	42	15
3	17	5	30	8	29	42	15
3	17	5	30	8	29	42	15
3	17	5	30	8	29	42	15
3	17	5	30	8	29	42	15
3	17	5	30	8	29	42	15
3	17	5	30	8	29	42	15
3	5	17	30	8	29	42	15

3 Pass 3 5 comparisons · swap $a[2] \leftrightarrow a[4]$

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
3	5	17	30	8	29	42	15
3	5	17	30	8	29	42	15
3	5	17	30	8	29	42	15
3	5	17	30	8	29	42	15
3	5	17	30	8	29	42	15
3	5	17	30	8	29	42	15
3	5	17	30	8	29	42	15
3	5	8	30	17	29	42	15

4 Pass 4 4 comparisons · swap $a[3] \leftrightarrow a[7]$

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
3	5	8	30	17	29	42	15
3	5	8	30	17	29	42	15
3	5	8	30	17	29	42	15
3	5	8	30	17	29	42	15
3	5	8	30	17	29	42	15
3	5	8	15	17	29	42	30

5 Pass 5 3 comparisons · no swap needed

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
3	5	8	15	17	29	42	30
3	5	8	15	17	29	42	30
3	5	8	15	17	29	42	30
3	5	8	15	17	29	42	30
3	5	8	15	17	29	42	30

6 Pass 6 2 comparisons · no swap needed

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
3	5	8	15	17	29	42	30
3	5	8	15	17	29	42	30
3	5	8	15	17	29	42	30

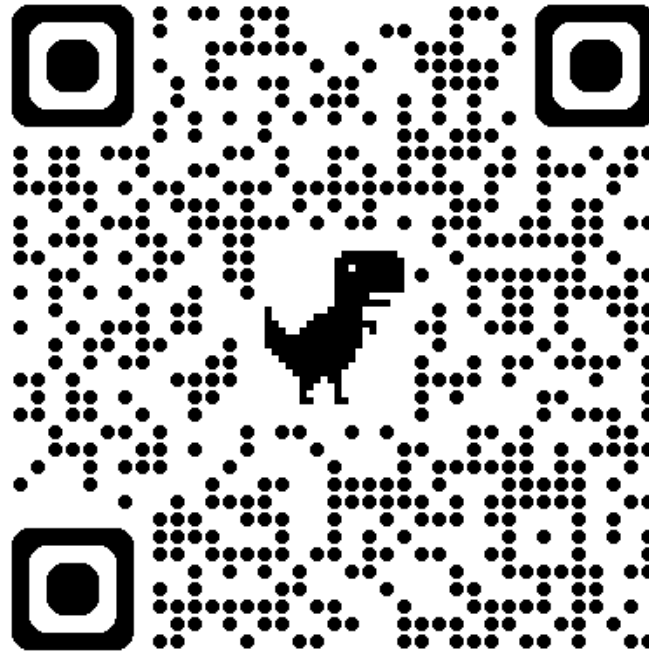
7 Pass 7 1 comparison · swap $a[6] \leftrightarrow a[7]$

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
3	5	8	15	17	29	42	30
3	5	8	15	17	29	30	42

Total: 28 comparisons · 5 swaps

Bubble Sort

Visualization 1



<https://noe-tec.github.io/data-structures/01-fundamentals/pages/sorting-walkthrough.html>

Selection Sort

The code

```
void selectionSort(int a[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        int minIndex = i;  
        for (int j = i + 1; j < n; j++) {  
            if (a[j] < a[minIndex]) {  
                minIndex = j;  
            }  
        }  
        if (minIndex != i) {  
            int tmp = a[i];  
            a[i] = a[minIndex];  
            a[minIndex] = tmp;  
        }  
    }  
}
```

- **Best case:**
 - $O(n^2)$
- **Worst case:**
 - $O(n^2)$

The inner loop only *remembers* where the minimum is. Nothing moves until the pass ends.

Bubble vs Selection Sort

Criteria	Bubble Sort (Optimized)	Selection Sort
Comparisons	$n - 1$ in the best case $n(n - 1)/2$ in the worst case	$n(n - 1)/2$ in any case
Swaps	0 in the best case $n(n - 1)/2$ in the worst case	$n - 1$ maximum
Time complexity	$O(n)$ in the best case $O(n^2)$ in the worst case	$O(n^2)$ in any case

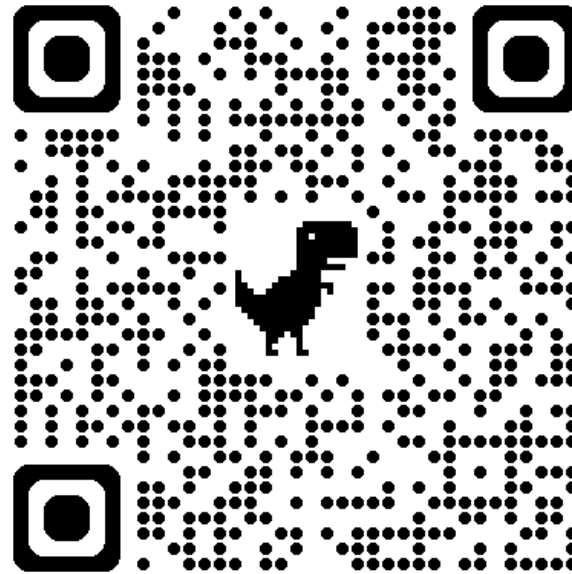
To take away

- Two algorithms can share a complexity class and still not be interchangeable.
- Big-O hides constants. Constants become real when elements get heavy.
- Stability is a property you only miss once you have lost it.

Visualizers

Sorting Algorithms

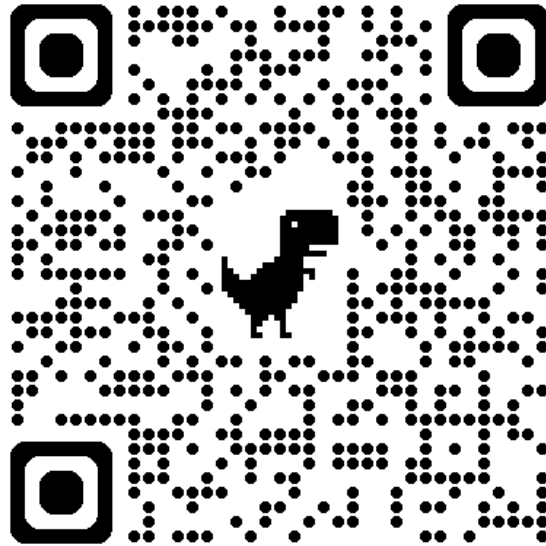
Visualization 3



<https://tobinatore.github.io/algovis/bubblesort.html>

Sorting Algorithms

Visualization 2



<https://www.hackerearth.com/practice/algorithms/sorting/bubble-sort/visualize/>

References



M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Boston, MA, USA: Pearson, 2014. Chapter 2.

V. M. de la Cueva Hernández, L. H. González Guerra, y E. G. Salinas Gurrión, *Estructuras de datos y algoritmos fundamentales*. Monterrey, México: Editorial Digital del Tecnológico de Monterrey, 2020.